

gen_optuna

April 1, 2026

```
[1]: %pip install scikit-learn pandas numpy optuna xgboost lightgbm catboost  
      ↪ imbalanced-learn
```

```
Requirement already satisfied: scikit-learn in ./venv/lib/python3.14/site-  
packages (1.8.0)  
Requirement already satisfied: pandas in ./venv/lib/python3.14/site-packages  
(3.0.1)  
Requirement already satisfied: numpy in ./venv/lib/python3.14/site-packages  
(2.4.3)  
Requirement already satisfied: optuna in ./venv/lib/python3.14/site-packages  
(4.8.0)  
Requirement already satisfied: xgboost in ./venv/lib/python3.14/site-packages  
(3.2.0)  
Requirement already satisfied: lightgbm in ./venv/lib/python3.14/site-packages  
(4.6.0)  
Requirement already satisfied: catboost in ./venv/lib/python3.14/site-packages  
(1.2.10)  
Requirement already satisfied: imbalanced-learn in ./venv/lib/python3.14/site-  
packages (0.14.1)  
Requirement already satisfied: scipy>=1.10.0 in ./venv/lib/python3.14/site-  
packages (from scikit-learn) (1.17.1)  
Requirement already satisfied: joblib>=1.3.0 in ./venv/lib/python3.14/site-  
packages (from scikit-learn) (1.5.3)  
Requirement already satisfied: threadpoolctl>=3.2.0 in  
./venv/lib/python3.14/site-packages (from scikit-learn) (3.6.0)  
Requirement already satisfied: python-dateutil>=2.8.2 in  
./venv/lib/python3.14/site-packages (from pandas) (2.9.0.post0)  
Requirement already satisfied: alembic>=1.5.0 in ./venv/lib/python3.14/site-  
packages (from optuna) (1.18.4)  
Requirement already satisfied: colorlog in ./venv/lib/python3.14/site-packages  
(from optuna) (6.10.1)  
Requirement already satisfied: packaging>=20.0 in ./venv/lib/python3.14/site-  
packages (from optuna) (26.0)  
Requirement already satisfied: sqlalchemy>=1.4.2 in ./venv/lib/python3.14/site-  
packages (from optuna) (2.0.48)  
Requirement already satisfied: tqdm in ./venv/lib/python3.14/site-packages  
(from optuna) (4.67.3)  
Requirement already satisfied: PyYAML in ./venv/lib/python3.14/site-packages
```

```

(from optuna) (6.0.3)
Requirement already satisfied: graphviz in ./venv/lib/python3.14/site-packages
(from catboost) (0.21)
Requirement already satisfied: matplotlib in ./venv/lib/python3.14/site-
packages (from catboost) (3.10.8)
Requirement already satisfied: plotly in ./venv/lib/python3.14/site-packages
(from catboost) (6.6.0)
Requirement already satisfied: six in ./venv/lib/python3.14/site-packages (from
catboost) (1.17.0)
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in
./venv/lib/python3.14/site-packages (from imbalanced-learn) (0.1.5)
Requirement already satisfied: Mako in ./venv/lib/python3.14/site-packages
(from alembic>=1.5.0->optuna) (1.3.10)
Requirement already satisfied: typing-extensions>=4.12 in
./venv/lib/python3.14/site-packages (from alembic>=1.5.0->optuna) (4.15.0)
Requirement already satisfied: MarkupSafe>=0.9.2 in ./venv/lib/python3.14/site-
packages (from Mako->alembic>=1.5.0->optuna) (3.0.3)
Requirement already satisfied: contourpy>=1.0.1 in ./venv/lib/python3.14/site-
packages (from matplotlib->catboost) (1.3.3)
Requirement already satisfied: cycycler>=0.10 in ./venv/lib/python3.14/site-
packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./venv/lib/python3.14/site-
packages (from matplotlib->catboost) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in ./venv/lib/python3.14/site-
packages (from matplotlib->catboost) (1.5.0)
Requirement already satisfied: pillow>=8 in ./venv/lib/python3.14/site-packages
(from matplotlib->catboost) (12.1.1)
Requirement already satisfied: pyparsing>=3 in ./venv/lib/python3.14/site-
packages (from matplotlib->catboost) (3.3.2)
Requirement already satisfied: narwhals>=1.15.1 in ./venv/lib/python3.14/site-
packages (from plotly->catboost) (2.18.1)

```

[notice] A new release of pip is

available: 26.0 -> 26.0.1

[notice] To update, run:

```
python -m pip install --upgrade pip
```

Note: you may need to restart the kernel to use updated packages.

```

[2]: # %%
import pandas as pd
import numpy as np
import warnings
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)
warnings.filterwarnings('ignore')

TRAIN_DATA = pd.read_csv('train-data.csv', index_col='id')

```

```

TRAIN_LABEL = pd.read_csv('train-label.csv', index_col='id')
TEST_DATA = pd.read_csv('test-data.csv', index_col='id')

if 'subscription' in TRAIN_DATA.columns:
    TRAIN_DATA = TRAIN_DATA.drop(columns=['subscription'])

def add_features(df):
    df = df.copy()
    df['prev_success'] = (df['poutcome'] == 'SUC').astype(int)
    df['never_contacted'] = (df['previous'] == 0).astype(int)
    df['pdays_clean'] = df['pdays'].apply(lambda x: 999 if x == -1 else x)
    df['prev_contacts_log'] = np.log1p(df['previous'])
    df['duration_log'] = np.log1p(df['duration'])
    df['log_balance'] = np.log1p(df['balance'].clip(lower=0))
    df['is_debt'] = (df['balance'] < 0).astype(int)
    df['log_campaign'] = np.log1p(df['campaign'])
    df['month_sin'] = np.sin(2 * np.pi * df['month'] / 12)
    df['month_cos'] = np.cos(2 * np.pi * df['month'] / 12)
    df['long_call'] = (df['duration'] > 300).astype(int)
    df['long_call_x_success'] = df['long_call'] * df['prev_success']
    df['duration_x_prev'] = df['duration_log'] * df['prev_contacts_log']
    return df

TRAIN_DATA = add_features(TRAIN_DATA)
TEST_DATA = add_features(TEST_DATA)
print('TRAIN_DATA shape:', TRAIN_DATA.shape)
print('TEST_DATA shape:', TEST_DATA.shape)

```

TRAIN_DATA shape: (29839, 29)

TEST_DATA shape: (19893, 29)

/Users/dayana/Desktop/3.1 / /lab/lab1/.venv/lib/python3.14/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm

```

[3]: # %%
from sklearn.preprocessing import OrdinalEncoder
from sklearn.model_selection import StratifiedKFold

cat_cols = ['job', 'marital_status', 'education', 'default_loan',
            'housing_loan', 'personal_loan', 'contact_type', 'poutcome']
num_cols = [c for c in TRAIN_DATA.columns if c not in cat_cols]

ENCODER = OrdinalEncoder(
    handle_unknown='use_encoded_value', unknown_value=-1
).fit(TRAIN_DATA[cat_cols])

```

```

def preprocess(df, encoder):
    df = df.copy()
    cat_enc = pd.DataFrame(
        encoder.transform(df[cat_cols]), columns=cat_cols, index=df.index
    )
    return pd.concat([cat_enc, df[num_cols].copy()], axis=1)

X_train = preprocess(TRAIN_DATA, ENCODER)
X_test = preprocess(TEST_DATA, ENCODER)
y_train = TRAIN_LABEL['subscription'].values

def target_encode_cv(X_tr, y_tr, X_te, cols, n_splits=5):
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    X_tr_enc = X_tr.copy()
    X_te_enc = X_te.copy()
    global_mean = y_tr.mean()
    for col in cols:
        oof = np.full(len(X_tr), global_mean)
        te_vals = np.zeros(len(X_te))
        for fold_tr_idx, fold_val_idx in skf.split(X_tr, y_tr):
            means = y_tr.iloc[fold_tr_idx].groupby(X_tr[col].iloc[fold_tr_idx]).
↪mean()
            oof[fold_val_idx] = X_tr[col].iloc[fold_val_idx].map(means).
↪fillna(global_mean).values
            te_vals += X_te[col].reset_index(drop=True).map(means).
↪fillna(global_mean).values / n_splits
        X_tr_enc[col + '_te'] = oof
        X_te_enc[col + '_te'] = te_vals
    return X_tr_enc, X_te_enc

y_series = pd.Series(y_train, index=X_train.index)
X_train_te, X_test_te = target_encode_cv(X_train, y_series, X_test, cat_cols)

print('X_train_te shape:', X_train_te.shape)
print('X_test_te shape:', X_test_te.shape)

```

```

X_train_te shape: (29839, 37)
X_test_te shape: (19893, 37)

```

```

[4]: # %%
import optuna
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import balanced_accuracy_score, roc_curve, make_scorer
from sklearn.ensemble import HistGradientBoostingClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

```

```

from catboost import CatBoostClassifier

# Settings
N_TRIALS = 60 # trials per model - increase for better results (costs time)
CV_SPLITS = 5 # inner CV folds for Optuna (faster than 10)
RANDOM_SEED= 42

scale_pos = (y_train == 0).sum() / (y_train == 1).sum()
X_arr = X_train_te.values
X_te_arr = X_test_te.values

inner_skf = StratifiedKFold(n_splits=CV_SPLITS, shuffle=True,
    random_state=RANDOM_SEED)

def youden_ba_cv(model, X, y, cv):
    """CV score using Youden J threshold (true BA maximiser)."""
    oof = np.zeros(len(y))
    for tr_idx, val_idx in cv.split(X, y):
        model.fit(X[tr_idx], y[tr_idx])
        oof[val_idx] = model.predict_proba(X[val_idx])[:, 1]
    fpr, tpr, ths = roc_curve(y, oof)
    j = tpr - fpr
    best_t = float(ths[np.argmax(j)])
    return balanced_accuracy_score(y, (oof >= best_t).astype(int))

#
# HGBM
#
def hgbm_objective(trial):
    params = {
        'learning_rate': trial.suggest_float('learning_rate', 0.005, 0.1,
    log=True),
        'max_iter': trial.suggest_int('max_iter', 300,
    2000),
        'max_leaf_nodes': trial.suggest_int('max_leaf_nodes', 10, 60),
        'max_depth': trial.suggest_int('max_depth', 3, 8),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 10, 150),
        'l2_regularization': trial.suggest_float('l2_regularization', 0.0, 10.
    0),
        'class_weight': 'balanced',
        'random_state': RANDOM_SEED,
        'early_stopping': False,
    }
    model = HistGradientBoostingClassifier(**params)
    return youden_ba_cv(model, X_arr, y_train, inner_skf)

print('Tuning HGBM...')

```

```

hgbm_study = optuna.create_study(direction='maximize',
                                sampler=optuna.samplers.
                                    ↪TPESampler(seed=RANDOM_SEED))
hgbm_study.optimize(hgbm_objective, n_trials=N_TRIALS, show_progress_bar=True)
best_hgbm = hgbm_study.best_params
print(f' Best HGBM BA: {hgbm_study.best_value:.5f}')
print(f' Params: {best_hgbm}')

#
# XGBoost
#
def xgb_objective(trial):
    params = {
        'n_estimators': trial.suggest_int( 'n_estimators', 200, ↪
        ↪1500),
        'learning_rate': trial.suggest_float('learning_rate', 0.005, 0.
        ↪1, log=True),
        'max_depth': trial.suggest_int( 'max_depth', 3, 8),
        'min_child_weight': trial.suggest_int( 'min_child_weight', 5, 100),
        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 1e-3, 10.
        ↪0, log=True),
        'reg_lambda': trial.suggest_float('reg_lambda', 1e-3, 10.
        ↪0, log=True),
        'gamma': trial.suggest_float('gamma', 0.0, 5.0),
        'scale_pos_weight': scale_pos,
        'eval_metric': 'logloss',
        'random_state': RANDOM_SEED,
        'n_jobs': -1,
    }
    model = XGBClassifier(**params)
    return youden_ba_cv(model, X_arr, y_train, inner_skf)

print('\nTuning XGBoost...')
xgb_study = optuna.create_study(direction='maximize',
                                sampler=optuna.samplers.
                                    ↪TPESampler(seed=RANDOM_SEED))
xgb_study.optimize(xgb_objective, n_trials=N_TRIALS, show_progress_bar=True)
best_xgb = xgb_study.best_params
print(f' Best XGB BA: {xgb_study.best_value:.5f}')
print(f' Params: {best_xgb}')

#
# LightGBM
#

```

```

def lgbm_objective(trial):
    num_leaves = trial.suggest_int('num_leaves', 10, 100)
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 200, 1500),
        'learning_rate': trial.suggest_float('learning_rate', 0.005, 0.1, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'num_leaves': num_leaves,
        'min_child_samples': trial.suggest_int('min_child_samples', 5, 100),
        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 1e-3, 10.0, log=True),
        'reg_lambda': trial.suggest_float('reg_lambda', 1e-3, 10.0, log=True),
        'class_weight': 'balanced',
        'random_state': RANDOM_SEED,
        'n_jobs': -1,
        'verbose': -1,
    }
    model = LGBMClassifier(**params)
    return youden_ba_cv(model, X_arr, y_train, inner_skf)

print('\nTuning LightGBM...')
lgbm_study = optuna.create_study(direction='maximize',
                                sampler=optuna.samplers.
                                TPESampler(seed=RANDOM_SEED))
lgbm_study.optimize(lgbm_objective, n_trials=N_TRIALS, show_progress_bar=True)
best_lgbm = lgbm_study.best_params
print(f' Best LGBM BA: {lgbm_study.best_value:.5f}')
print(f' Params: {best_lgbm}')

#
# CatBoost
#
def cat_objective(trial):
    params = {
        'iterations': trial.suggest_int('iterations', 300, 2000),
        'learning_rate': trial.suggest_float('learning_rate', 0.005, 0.1, log=True),
        'depth': trial.suggest_int('depth', 3, 10),
    }

```

```

        'l2_leaf_reg':      trial.suggest_float('l2_leaf_reg', 1.0, 15.0),
        'bagging_temperature': trial.suggest_float('bagging_temperature', 0.0, 1.
↪0),
        'random_strength':  trial.suggest_float('random_strength', 0.0, 5.0),
        'auto_class_weights': 'Balanced',
        'eval_metric':      'Logloss',
        'random_seed':      RANDOM_SEED,
        'verbose':          0,
    }
    model = CatBoostClassifier(**params)
    return youden_ba_cv(model, X_arr, y_train, inner_skf)

print('\nTuning CatBoost...')
cat_study = optuna.create_study(direction='maximize',
                               sampler=optuna.samplers.
↪TPESampler(seed=RANDOM_SEED))
cat_study.optimize(cat_objective, n_trials=N_TRIALS, show_progress_bar=True)
best_cat = cat_study.best_params
print(f' Best CAT BA: {cat_study.best_value:.5f}')
print(f' Params: {best_cat}')

print('\n' + '='*50)
print('OPTUNA SUMMARY')
print('='*50)
print(f' HGBM : {hgbm_study.best_value:.5f}')
print(f' XGB  : {xgb_study.best_value:.5f}')
print(f' LGBM : {lgbm_study.best_value:.5f}')
print(f' CAT  : {cat_study.best_value:.5f}')

```

Tuning HGBM...

Best trial: 56. Best value: 0.869263: 100%| | 60/60 [21:51<00:00, 21.86s/it]

Best HGBM BA: 0.86926

Params: {'learning_rate': 0.031745219196835026, 'max_iter': 1662, 'max_leaf_nodes': 12, 'max_depth': 8, 'min_samples_leaf': 43, 'l2_regularization': 8.729921546982613}

Tuning XGBoost...

Best trial: 42. Best value: 0.871754: 100%| | 60/60 [06:54<00:00, 6.90s/it]

Best XGB BA: 0.87175

Params: {'n_estimators': 930, 'learning_rate': 0.01711298021922337, 'max_depth': 8, 'min_child_weight': 19, 'subsample': 0.9144905105460359, 'colsample_bytree': 0.9687392914246111, 'reg_alpha': 0.005704849589871292, 'reg_lambda': 0.10198791505594093, 'gamma': 0.5340772759198125}

Tuning LightGBM...

Best trial: 32. Best value: 0.873116: 100%| | 60/60 [27:10<00:00, 27.17s/it]

Best LGBM BA: 0.87312

Params: {'num_leaves': 95, 'n_estimators': 538, 'learning_rate': 0.08154366958417066, 'max_depth': 9, 'min_child_samples': 74, 'subsample': 0.8559039880578655, 'colsample_bytree': 0.6510178477019677, 'reg_alpha': 8.811781685059318, 'reg_lambda': 0.09599175577640591}

Tuning CatBoost...

Best trial: 7. Best value: 0.87144: 100%| | 60/60 [28:54<00:00, 28.92s/it]

Best CAT BA: 0.87144

Params: {'iterations': 358, 'learning_rate': 0.07621195864233186, 'depth': 5, 'l2_leaf_reg': 10.275311980955747, 'bagging_temperature': 0.31171107608941095, 'random_strength': 2.600340105889054}

=====

OPTUNA SUMMARY

=====

HGBM : 0.86926
XGB : 0.87175
LGBM : 0.87312
CAT : 0.87144

```
[5]: # %%  
from sklearn.model_selection import StratifiedKFold  
from sklearn.metrics import balanced_accuracy_score, roc_curve  
  
N_SPLITS = 10  
skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=42)  
  
model_names = ['HGBM', 'XGB', 'LGBM', 'CAT']  
oof_preds = {name: np.zeros(len(y_train)) for name in model_names}  
test_preds = {name: np.zeros(len(X_test_te)) for name in model_names}  
  
for fold, (tr_idx, val_idx) in enumerate(skf.split(X_arr, y_train)):  
    X_tr, X_val = X_arr[tr_idx], X_arr[val_idx]  
    y_tr, y_val = y_train[tr_idx], y_train[val_idx]  
  
    # HGBM  
    m = HistGradientBoostingClassifier(  
        **best_hgbm, class_weight='balanced',
```

```

        random_state=RANDOM_SEED, early_stopping=False
    )
    m.fit(X_tr, y_tr)
    oof_preds['HGBM'][val_idx] = m.predict_proba(X_val)[: , 1]
    test_preds['HGBM'] += m.predict_proba(X_te_arr)[: , 1] / N_SPLITS

# XGB
xgb_p = {**best_xgb,
         'scale_pos_weight': scale_pos,
         'eval_metric': 'logloss',
         'random_state': RANDOM_SEED, 'n_jobs': -1}
m = XGBClassifier(**xgb_p)
m.fit(X_tr, y_tr)
oof_preds['XGB'][val_idx] = m.predict_proba(X_val)[: , 1]
test_preds['XGB'] += m.predict_proba(X_te_arr)[: , 1] / N_SPLITS

# LGBM
lgbm_p = {**best_lgbm,
         'class_weight': 'balanced',
         'random_state': RANDOM_SEED, 'n_jobs': -1, 'verbose': -1}
m = LGBMClassifier(**lgbm_p)
m.fit(X_tr, y_tr)
oof_preds['LGBM'][val_idx] = m.predict_proba(X_val)[: , 1]
test_preds['LGBM'] += m.predict_proba(X_te_arr)[: , 1] / N_SPLITS

# CatBoost
cat_p = {**best_cat,
         'auto_class_weights': 'Balanced',
         'eval_metric': 'Logloss',
         'random_seed': RANDOM_SEED, 'verbose': 0}
m = CatBoostClassifier(**cat_p)
m.fit(X_tr, y_tr)
oof_preds['CAT'][val_idx] = m.predict_proba(X_val)[: , 1]
test_preds['CAT'] += m.predict_proba(X_te_arr)[: , 1] / N_SPLITS

print(f'Fold {fold+1}/{N_SPLITS} done')

print('\nOOB Balanced Accuracy per model (Youden J threshold):')
for name in model_names:
    fpr, tpr, thresholds = roc_curve(y_train, oof_preds[name])
    j = tpr - fpr
    best_t = float(thresholds[np.argmax(j)])
    best_ba = balanced_accuracy_score(
        y_train, (oof_preds[name] >= best_t).astype(int)
    )
    print(f' {name:5s}: BA={best_ba:.5f} threshold={best_t:.4f}')

```

Fold 1/10 done

Fold 2/10 done
Fold 3/10 done
Fold 4/10 done
Fold 5/10 done
Fold 6/10 done
Fold 7/10 done
Fold 8/10 done
Fold 9/10 done
Fold 10/10 done

OOF Balanced Accuracy per model (Youden J threshold):

HGBM : BA=0.87119 threshold=0.4224
XGB : BA=0.87346 threshold=0.3197
LGBM : BA=0.87505 threshold=0.3816
CAT : BA=0.87157 threshold=0.4349

```
[6]: # %%  
from sklearn.model_selection import StratifiedKFold  
from sklearn.metrics import balanced_accuracy_score, roc_curve  
from lightgbm import LGBMClassifier  
from scipy.stats import rankdata  
  
# Rank-normalise each model's OOF & test probs  
def rank_norm(arr):  
    return rankdata(arr) / len(arr)  
  
oof_rank = np.column_stack([rank_norm(oof_preds[n]) for n in model_names])  
test_rank = np.column_stack([rank_norm(test_preds[n]) for n in model_names])  
  
# LGBM meta-model trained on OOF only (no leakage)  
meta_skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=99)  
oof_stack = np.zeros(len(y_train))  
  
meta_params = dict(  
    n_estimators=300, learning_rate=0.03, num_leaves=15,  
    min_child_samples=20, class_weight='balanced',  
    random_state=42, n_jobs=-1, verbose=-1,  
)  
  
for fold, (tr_idx, val_idx) in enumerate(meta_skf.split(oof_rank, y_train)):  
    meta = LGBMClassifier(**meta_params)  
    meta.fit(oof_rank[tr_idx], y_train[tr_idx])  
    oof_stack[val_idx] = meta.predict_proba(oof_rank[val_idx])[:, 1]  
  
final_meta = LGBMClassifier(**meta_params)  
final_meta.fit(oof_rank, y_train)  
test_stack = final_meta.predict_proba(test_rank)[:, 1]
```

```

# BA-weighted average as second ensemble signal
oof_ba_scores = {}
for name in model_names:
    fpr, tpr, ths = roc_curve(y_train, oof_preds[name])
    j = tpr - fpr
    t = float(ths[np.argmax(j)])
    oof_ba_scores[name] = balanced_accuracy_score(
        y_train, (oof_preds[name] >= t).astype(int)
    )

weights = np.array([oof_ba_scores[n] for n in model_names])
weights = (weights - weights.min()) / (weights.max() - weights.min() + 1e-9)
weights /= weights.sum()

print('Model weights (BA-proportional):')
for n, w in zip(model_names, weights):
    print(f' {n}: {w:.4f}')

oof_wavg = oof_rank @ weights
test_wavg = test_rank @ weights

# Blend meta + weighted average
BLEND_W = 0.6
oof_blend = BLEND_W * oof_stack + (1 - BLEND_W) * oof_wavg
test_blend = BLEND_W * test_stack + (1 - BLEND_W) * test_wavg

# Youden J threshold
fpr_b, tpr_b, thresholds_b = roc_curve(y_train, oof_blend)
j_b = tpr_b - fpr_b
best_threshold = float(thresholds_b[np.argmax(j_b)])
best_ba = balanced_accuracy_score(
    y_train, (oof_blend >= best_threshold).astype(int)
)

print(f'\nBlended OOF BA : {best_ba:.5f}')
print(f'Optimal threshold (Youden J): {best_threshold:.4f}')

# Threshold neighbourhood
print('\nThreshold | #Pred-1 | OOF BA')
print('-' * 38)
for t in np.arange(
    max(0.01, best_threshold - 0.04),
    min(0.99, best_threshold + 0.05),
    0.005
):
    preds = (oof_blend >= t).astype(int)

```

```

ba    = balanced_accuracy_score(y_train, preds)
mark  = ' <- best' if abs(t - best_threshold) < 0.003 else ''
print(f' {t:.3f} | {preds.sum():6d} | {ba:.4f}{mark}')

```

Model weights (BA-proportional):

```

HGBM: 0.0000
XGB: 0.3484
LGBM: 0.5928
CAT: 0.0588

```

Blended OOF BA : 0.87389

Optimal threshold (Youden J): 0.5946

Threshold | #Pred-1 | OOF BA

```

-----
0.555 | 7779 | 0.8728
0.560 | 7714 | 0.8733
0.565 | 7635 | 0.8732
0.570 | 7584 | 0.8727
0.575 | 7523 | 0.8726
0.580 | 7462 | 0.8731
0.585 | 7380 | 0.8732
0.590 | 7278 | 0.8733
0.595 | 7197 | 0.8739 <- best
0.600 | 7112 | 0.8731
0.605 | 7037 | 0.8732
0.610 | 6990 | 0.8736
0.615 | 6942 | 0.8732
0.620 | 6895 | 0.8734
0.625 | 6854 | 0.8736
0.630 | 6805 | 0.8734
0.635 | 6755 | 0.8735
0.640 | 6697 | 0.8731
0.645 | 6642 | 0.8726

```

```

[7]: # %%
test_classes = (test_blend >= best_threshold).astype(int)

n1 = test_classes.sum()
n0 = (test_classes == 0).sum()
print(f'Prediction distribution - 0: {n0}, 1: {n1} '
      f'(pos-rate {n1/(n0+n1)*100:.1f}%)')
print(f'Blended OOF BA : {best_ba:.5f}')
print(f'Threshold used : {best_threshold:.4f}')

```

Prediction distribution - 0: 15052, 1: 4841 (pos-rate 24.3%)

Blended OOF BA : 0.87389

Threshold used : 0.5946

```
[8]: # %%  
submission = pd.DataFrame({  
    'id': TEST_DATA.index,  
    'subscription': test_classes  
})  
submission.to_csv('submission_optuna.csv', index=False)  
print('Saved! Preview:')  
print(submission.head())  
print(submission['subscription'].value_counts())
```

Saved! Preview:

	id	subscription
0	37797	0
1	37798	0
2	37799	0
3	37800	0
4	37801	1

subscription

0 15052

1 4841

Name: count, dtype: int64